

28.04.25

This week, I must make my environments more complex, introduce several agents and also align the input / output formats with the standard (it's very close already).

1. Change input / output format to be in accordance with the “standard” defined by Kep (otherwise Python wrappen won't digest it).

NOTE: one advantage of 2d positional encoding is that we don't need to create the size atom.

Let's reverse-engineer their environment encoding:

```
train(0) .
start(0, (6,5), 0, s) .
end(0, (24,7), 32) .

train(1) .
start(1, (24,6), 2, n) .
end(1, (6,8), 34) .
cell((0,0), 0) .
cell((0,1), 0) .
cell((0,2), 0) .
cell((0,3), 0) .
cell((0,4), 0) .
cell((0,5), 0) .
```

Let's decipher these atoms by turning their numeric values back to meaningful variable names:

```
train(TRAIN_NUMBER) .
start(TRAIN_NUMBER, (X_POS,Y_POS), START_TIME, DIRECTION) .
end(TRAIN_NUMBER, (X_POS,Y_POS), END_TIME) .
cell((X_POS, Y_POS), SEGMENT_TYPE) .
```

This seems to be all, doesn't it? Let's try to translate my routes into this format:

My encoding	Translated
start(0) . end(6) . grid_size(3,3) . segment(0, 128) . segment(1, 0) .	train(1) . start(1, (0, 0), 0, NORTH) . end(1, (0, 2), 3) . cell((0, 0), 32800) . cell((1, 0), 0) . cell((2, 0), 0) .

segment(2, 0). segment(3, 32800). segment(4, 0). segment(5, 0). segment(6, 8192). segment(7, 0). segment(8, 0).	cell((0, 1), 32800). cell((1, 1), 0). cell((2, 1), 0). cell((0, 2), 32800). cell((1, 2), 0). cell((2, 2), 0).
---	--

NOTE: we also have no deadends, so I need to use **other types** at the ends of my routes.

QUESTION: is there a script to convert matrix representation of an environment into asp facts?

NOTE: I will need to revise the logic of max-time for several trains - perhaps by taking the latest train arrival as the upper bound or by setting up a separate timer for each train.

QUESTION: What is the required output format?

Let's reinvent the constraints in this new formatting:

Constraint Idea	ASP phrasing
A train must stay on the track (a new one, since in the previous formatting it was included in the generator).	There must be no answer set where the train is on the 0 type of segment.
At the same moment of time, there must be only one action.	There must be no answer set where there are two different actions at the same time.
The train must start from the departure station.	There must be no answer set where the train doesn't start from the departure station.
The train must end at the destination station.	There must be no answer set where the train doesn't arrive at the destination.
If there is no junction, the action must only be "move forward" or "wait".	There must be no answer set that contains "dispatcher" atoms where the type... This is already implied by how "dispatcher" atoms are generated.

INTUITION: bind cells and possible actions. For example, by introducing an atom "possible actions (position, actions)". Then use these atoms for a more targeted generation.

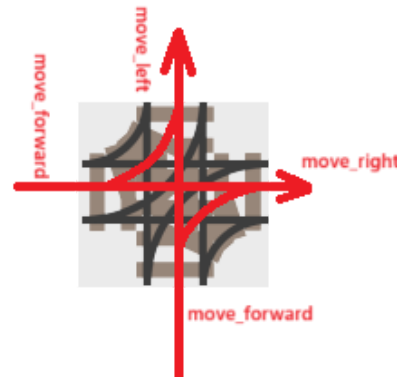
OBSERVATION: In ASP, different naming of variables does not imply difference in values - it must be indicated explicitly.

INTUITION: use my previous encoding, but wrap it in the new format;



How should a train move on these? What are the possible directions? Let's draw.

What I see in how the segment can be passed is that the choice depends on the direction the segment is entered.



There are two alternatives: to put all possible actions into one tuple or to split them into two sub-tuple. While the second seems more structured, the first might still work - I'll only understand it later.

Describe all track types. **DONE (but can be improved further).**

INTUITION: I must encode action → coordinate change mapping, as it was in my original encoding.

I'm now operating in a unit file. Let's keep it as simple as possible:

- A generator randomly produces actions (one of four possible at a time; first, even *without track types* - just to understand the geometry of movement).
- The function takes in the action and produces the position.

It's harder than I thought: without track types / directions, following the action "move_forward" can lead to either of the neighbouring cells.

INTUITION: direction must be reassigned to each "dispatcher" atom.

IDEA: What if I convert 2D positional encoding into unique integer encoding (1D)?

I think this idea is correct - we did the same / read about it in the RL course. It also appeals more to how ASP works - tuples feel alien there.

Is there a formula to convert 2d into 1d? Let's think graphically:

Yes, the intuition is to merge Y and X coordinates into one number → sort them in ascending order → assign 0 based integer indexes with step 1.

For now, I have obtained this:

```
position(0) position(1) position(2) position(10) position(11)
position(12) position(20) position(21) position(22)
```

Is it convenient to track movement with these integers?

Let's say, the agent is in position 0 and takes action "north". What cell will it move to?

INTUITION: count up to the number of cells;

Okay, no I have mapping to the 1d indexes. What's next?

INTUITION: the grid size is the count of unique X and unique Y. **Correct. Done.**

INTUITION: expand the "cell_actions" atom by "direction" variable - so that the choice of actions depends on the direction the train is following.

30.04.25

It seems reasonable to bind three variables: type, train entry direction and change of position ("delta" hereafter). Before attempting to encode it in asp, let's visualise it for clarity as a table:

https://docs.google.com/spreadsheets/d/1vVDptAVsx0L1_bZe3m-rT9NVDgNmR9R0W8ZQWghZTlw/edit?gid=0#gid=0

NOTE: once the table is done, I need to group rows by actions; same actions - one line in asp.

01.05.25

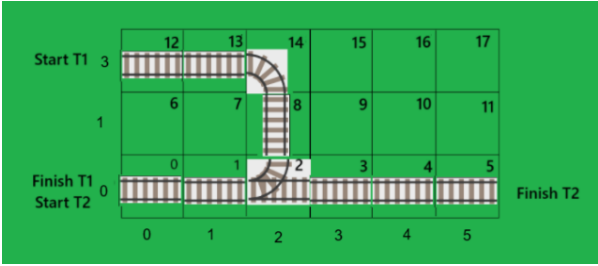
Now I have this table that maps direction, type of track and the possible directions (some deterministic, some with stochastic). What's next?

Now, I'll need to wrap this into an encoding.

INTUITION: a train() atom must carry its latest direction along / it should be contained in the "dispatcher" atom.

Let's now focus on representing this mapping between type, direction and positional delta.

Let's also test the **2d to 1d encoding converter** by running it on all the existing grids.

Grid	Conversion Correctness
Straight line	Yes
Г-turn	Yes
П-turn	Yes
Single switch (a typo: 3 must be 2) 	Yes
"Real" task environment: 30x30, 2 agents;	No, but let's leave it for later. Perhaps, there can be grounding issues . HYPOTHESIS: when binary coordinates exceed 9 (each), my encoding fails due to *10 multiplier.

OBSERVATION: in asp, CAPITAL strings are variables inside atoms, and lowercase strings are constants.

Good. Now I have the correctly working positional converter that I can work with. **Solved**.

ALERT: we need a new name since there were some "Gnomes" already! Let us be '**die Zauberbahn**'.

What's next? Next, I need to combine the **position converter** and **type-direction-delta** module to solve **one-train deterministic tracks**.

DISCOVERY: clingo natively supports multi file programs! The syntax is like this:

```
clingo base.lp grid.lp rules.lp
```

So, in my case it will be something like:

```
clingo 0 2d_to_1d_positions.lp type_direction_delta.lp
```

INTUITION: delta mapping is better done using negative statements, i.e. constraints.

Let's say I have this output of the converter:

```
cell_flat(0,32800) cell_flat(1,0) cell_flat(2,0) cell_flat(3,32800)
cell_flat(4,0) cell_flat(5,0) cell_flat(6,32800) cell_flat(7,0)
cell_flat(8,0) grid_size(3,3)
```

What's next? I'm in the generation stage, so I must focus on producing everything possible.

For each of these atoms, I want to generate possible movements. Do I have everything necessary for this? (let's start very small and naive).

Let's say we have four possible directions of the train:

```
directions(north; south; east; west).
```

The obtained new atoms must contain: position, direction. The type is no longer necessary because we have enough information.

Let's create "tracker" atoms that are responsible for holding position and direction - for now, just for a single train.

```
tracker(POSITION, DIRECTION).
```

Let's look at the output and think what we might do next:

```
Answer: 1
tracker(0,north) tracker(1,north) tracker(2,north) tracker(3,north) tracker(4,north)
tracker(5,north) tracker(6,north) tracker(7,north) tracker(8,north) tracker(0,south)
tracker(1,south) tracker(2,south) tracker(3,south) tracker(4,south) tracker(5,south)
tracker(6,south) tracker(7,south) tracker(8,south) tracker(0,east) tracker(1,east)
tracker(2,east) tracker(3,east) tracker(4,east) tracker(5,east) tracker(6,east)
tracker(7,east) tracker(8,east) tracker(0,west) tracker(1,west) tracker(2,west)
tracker(3,west) tracker(4,west) tracker(5,west) tracker(6,west) tracker(7,west)
tracker(8,west)
```

The thing is that in each of these positions (first variable), some actions are impossible: for example, at 0, the train cannot go east.

Does it make sense to include time now? Not really, let's deal only with position vs possible direction problem.

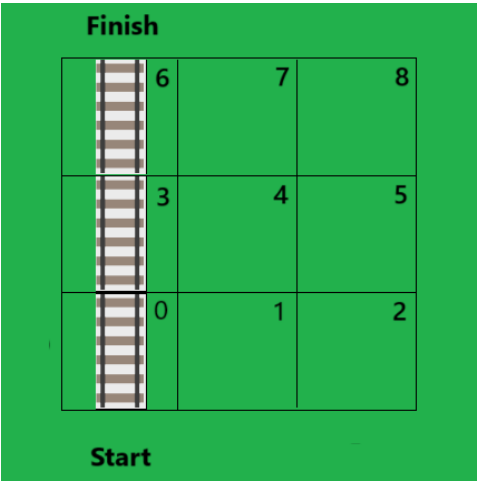
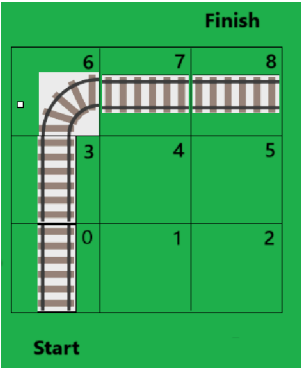
Intuitively, there are two ways to achieve this: through generation and through constraints. The better one is the one that causes less “grounding” in ASP and that also is shorter to write. Let’s try both.

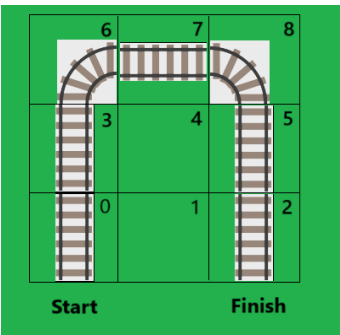
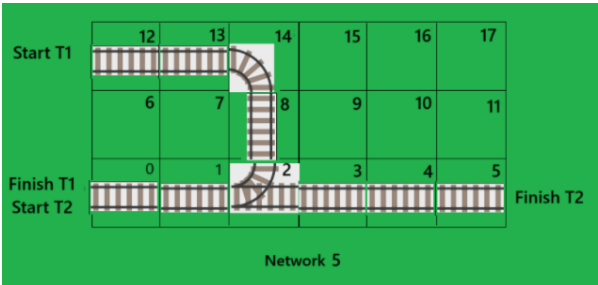
Let’s go for constraints first:

Constraint Idea	ASP phrasing
If the track is directed south-north, the train can't be moving east / west.	There must be no answer set where the train moves east or west on a track 32800.

OBSERVATION: it seems that conditional generation could be better. The problem with constraints is that they prohibit THE WHOLE answer set, including the correct atoms.

It’s still very naive, since I don’t consider the entry direction of the cell, but at least I remove the impossible direction. Let’s track the outputs:

Track	Answer
	<pre> tracker(0,south) tracker(3,south) tracker(6,south) tracker(0,north) tracker(3,north) tracker(6,north) </pre>
	<pre> tracker(6,north) tracker(6,west) tracker(7,west) tracker(8,west) tracker(7,east) tracker(8,east) tracker(0,south) tracker(3,south) tracker(0,north) tracker(3,north) </pre>

	<pre>tracker(6,north) tracker(6,west) tracker(8,north) tracker(8,east) tracker(7,west) tracker(7,east) tracker(0,south) tracker(1,south) tracker(3,south) tracker(5,south) tracker(0,north) tracker(1,north) tracker(3,north) tracker(5,north)</pre>
	<pre>tracker(2,west) tracker(2,east) tracker(2,south) tracker(14,north) tracker(14,east) tracker(0,west) tracker(1,west) tracker(3,west) tracker(4,west) tracker(5,west) tracker(12,west) tracker(13,west) tracker(0,east) tracker(1,east) tracker(3,east) tracker(4,east) tracker(5,east) tracker(12,east) tracker(13,east) tracker(8,south) tracker(8,north)</pre>

Good. What's the next step?

03.05.25

Let's revise what I've done. Now I have a set of atoms called "tracker" (an extremely intuitive name for me). I've tested if the **tracker** currently only contains viable combinations of train position - direction (they are dependent variables). The test was successful on all simple grids.

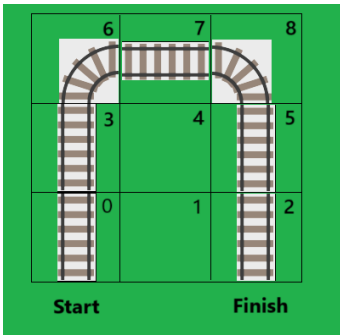
What's next?

Next, I want to extend the **tracker** atoms to include:

Possible Extension	Reasoning
Add cell leaving direction beyond entry one	What will it give me?
Add a train to each	With only 1 train, it doesn't change anything. With more, it's critical.
Add moment of time when the train occupies the cell	It's critical for the task no matter how many trains there are

Add positional delta	Will let me 'chain' answers, using a constraint to remove those that don't go in a sequence
Add next position	A more readable substitute for positional delta

Let's focus on deterministic single-train Π -turn to settle this.



Does this answer set contain the right answer (the route)?

```

tracker(6,north) tracker(6,west) tracker(8,north)
tracker(8,east) tracker(7,west) tracker(7,east)
tracker(0,south) tracker(1,south) tracker(3,south)
tracker(5,south) tracker(0,north) tracker(1,north)
tracker(3,north) tracker(5,north)

```

(let's mark the correct track as bold purple)

Where is 2?! Solution: there was a mistake in my 2d encoding.

Let's check the new answer for the correct path:

```

tracker(6,north) tracker(6,west) tracker(8,north) tracker(8,east) tracker(7,west)
tracker(7,east) tracker(0,south) tracker(2,south) tracker(3,south) tracker(5,south)
tracker(0,north) tracker(2,north) tracker(3,north) tracker(5,north)

```

Yes, now we have the full right track in. How do we remove the wrong positions?

Before I go to constraints, let's extend the tracker atom to include the leaving direction and/or positional delta (the two together may be redundant).

To make it faster, let's limit to the track types we observe in the Π -turn.

NOTE: I must remember that in my positional table there might be mistakes and keep checking it every time I address it.

Let's see the output of the new encoding:

```

tracker(6,north,7) tracker(6,west,3) tracker(8,north,7) tracker(8,east,5)
tracker(7,west,6) tracker(7,east,8) tracker(0,south,-3) tracker(2,south,-1)
tracker(3,south,0) tracker(5,south,2) tracker(0,north,3) tracker(2,north,5)
tracker(3,north,6) tracker(5,north,8)

```

There are immediately suspicious negative positions - where do they come from?

H1: they come from the train going out of the grid. **Confirmed:** in positions 0 and 2 there is no opportunity to move south because they are dead ends. Let's solve it later (through constraints

or at generation time). For now, let's explore the rest of the answer set. Is there the correct path? Let's find it and mark it as bold green.

INTUITION: I must produce atoms that are extremely readable and intuitive. Tracker seems to be such.

Adding the condition of the next cell being presented solved the problem of negative positions:

```
tracker(6,north,7) tracker(6,west,3) tracker(8,north,7) tracker(8,east,5)  
tracker(7,west,6) tracker(7,east,8) tracker(3,south,0) tracker(5,south,2)  
tracker(0,north,3) tracker(2,north,5) tracker(3,north,6) tracker(5,north,8)
```

The right path is still here. It seems intuitively correct to swap current position and entry direction, so that it will look like:

```
tracker(north, 6, 7) .
```

And add the outgoing direction to tracker:

```
tracker(north, 6, east, 7) .
```

INTUITION: I've built a kind of *bigram* with this atom. And also the whole task is about sequences, in a sense.

Let's test new encoding on the Π -track:

```
tracker(north,6,east,7) tracker(west,6,south,3) tracker(north,8,west,7)  
tracker(east,8,south,5) tracker(west,7,west,6) tracker(east,7,east,8)  
tracker(south,3,south,0) tracker(south,5,south,2) tracker(north,0,north,3)  
tracker(north,2,north,5) tracker(north,3,north,6) tracker(north,5,north,8)
```

Let's find the presence of the correct track.

CONCERN: I don't use control actions at all.

ANSWER: I don't need any actions on deterministic tracks! They will appear together with switches. If I needed to, I could just assign 'move_forward' to be in every tracker atom.

The current encoding is incredibly readable and intuitive. I'm satisfied. Let's build on.

INTUITION: we can chain tracker atoms that belong to one route by pairing each output with each entry direction / cell. Let's single out the correct route:

```
tracker(north,0,north,3) tracker(north,3,north,6) tracker(north,6,east,7)  
tracker(east,7,east,8) tracker(east,8,south,5) tracker(south,5,south,2)
```

NOTE: I also accidentally implemented train stopping where the track ends by putting the condition of the next cell being present!

OBSERVATION: the remaining atoms are basically just travelling back.

INTUITION: this chaining is easier to express as a constraint.

Constraint Idea	ASP phrasing
All tracker atoms in the answer set must be coupled through their last / first pairs of variables.	There must be no answer set where there are two tracker atoms (INT: it will prohibit <i>all atoms</i>) not coupled.

Let's set this aside for a while. Given that the remaining atoms are essentially just *the road back*, it's easier to prohibit atoms that don't lead from the given departure point to the given destination.

Constraint Idea	ASP phrasing
The train must arrive at the destination.	There must be no answer set where there is no tracker atom whose second position is not the destination (it will likely prohibit the whole answer, but let's try).

For this, let's also add flattened start and end cells. Let's test it on deterministic single-train environments:

straight_line +
Γ-turn +
Π-turn +

And the single switch +

Now back to the constraint.

INTUITION: it's methodologically wrong to not generate path atoms with a choice rule.

LLM: no, it's sound as long as I'm describing the environment.

INTUITION: rename "tracker" into "path" - because what I essentially obtained is a very exact path description!

INTUITION: no choice rule generation - no constraints!

INTUITION: use the obtained tracker ('path') atoms in *generation*.

LLM: *Exactly — that's spot on.*

Your *tracker/4* atoms **form a declarative map of the railway network**:

- They describe **how track segments connect** (via directions and positions).
- They encode **valid transitions**, abstracting away raw grid coordinates or hardcoded conditionals.
- This is the **static world model** — the infrastructure the trains will operate on.

That gives you a powerful foundation: now you can build **dynamic reasoning** (like train movements, scheduling, conflict detection) on top of this clean, constraint-ready layout.

INTUITION: in asp, constraints are to manage the intentional combinatorial explosion caused by a choice rule. Without a choice rule, there is no need for constraints - everything that happens is just expressing the input facts through other atoms.

LLM: “Yes — *that's an **excellent intuition**, and it captures the heart of how ASP works.*”

What's the difference between these syntaxes:

- 1.
- 2.
- 3.

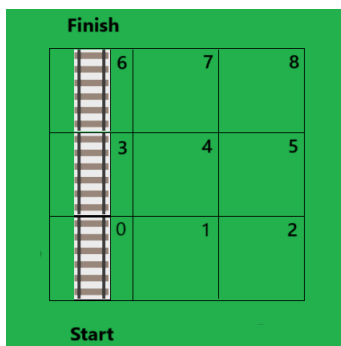
EXPLORATION: read on choice role scoping.

Let's explore the output to understand how to move further / what to limit.

Example answer:

```
move(1,1,0,3) move(1,3,0,3) move(1,4,0,3) move(1,1,2,5) move(1,2,2,5)
move(1,3,2,5) move(1,1,3,6)
```

Let's start with an even simpler case of a straight line:



Sample answer:

```
transition(1,3,3,0) transition(1,2,6,3)
transition(1,3,6,3)
```

Well, at least **all separate transitions are legit.**

Now it also seems that I can implement my constraint of pairwise coupling: there must be no answer set where there are two “transition” atoms whose (let’s look at the right one:

```
transition(1,1,0,3) transition(1,2,3,6)
```

)

... no, it will kill all answers again!

05.05.25

Constraint Idea	ASP phrasing	Code
Moments of time must not repeat	There must be no answer set where there are two different “transition” atoms for the same train and the same moment of time.	<pre>:- transition(TRAIN_1, MOMENT_1, _, _), transition(TRAIN_2, MOMENT_2, _, _), TRAIN_1 == TRAIN_2, MOMENT_1 == MOMENT_2.</pre>
Transitions must not repeat	There must be no answer set where there are two ‘transition’ atoms with different moments of time but the same transition for the same train (would prohibit circular movement).	<pre>:- transition(TRAIN_1, MOMENT_1, FROM_CELL, TO_CELL), transition(TRAIN_2, MOMENT_2, FROM_CELL, TO_CELL), TRAIN_1 == TRAIN_2, MOMENT_1 != MOMENT_2.</pre>
Train must not jump between different cells	There must be no answer set where there are two transition atoms where the moment of time is +1 but the transition is nonconsecutive (let’s phrase this more precisely: <pre>transition(1,1,7,8) transition(1,2,6,7)</pre> So, there must be no two transition atoms where the train is the same, the moment of time is + 1 but last variable of the first and the pre-last variable of the second are different)	<pre>:- transition(TRAIN, MOMENT, _, TO_CELL), transition(TRAIN, MOMENT+1, FROM_CELL, _), TO_CELL != FROM_CELL.</pre>
The train must not go back	There must be no two ‘transition’ atoms where the second one holds the moment of time + 1, but <pre>transition(1,1,0,3)</pre>	<pre>:- transition(TRAIN, MOMENT, FROM_CELL, TO_CELL), transition(TRAIN, MOMENT+1, TO_CELL, FROM_CELL).</pre>

	<code>transition(1,2,3,0)</code>	
The train must arrive at the destination Result: works perfectly on deterministic environments; interesting to explore on 1 switch env.	The 3rd and 4th variables are mutually swapped.	
The train must start from the departure station	There must be no answer set where for the moment 1 (better say: at the <i>indicated departure time!</i>) there is no 'transition' atom that holds 0 as the TO_CELL value.	<pre>:- transition(TRAIN, DEPARTURE_TIME, FLAT_POS, _), start_flat(TRAIN, FLAT_POS, DEPARTURE_TIME, _).</pre>
The train must arrive at the final station.	There must be no answer set where at the (pre-) last moment of time there is no transition to the respective destination station by the given train.	<pre>:- end_flat(TRAIN, FLAT_POS, ARRIVAL_TIME), transition(TRAIN, FLAT_POS, ARRIVAL_TIME - 1).</pre>